第二次挑战：前缀缓存与请求流程

SGLang 0.5.14 | Ray 2.56.0 | Qwen/Qwen3-0.6B

任务一：实验设计

设备为 NVIDIA GeForce RTX 5060 Laptop GPU（8151 MiB）；单卡 TP=1，float16，context-length=4096，mem-fraction-static=0.72，max-running-requests=8。服务日志显示 RadixCache 已启用，page-size=1；使用原生 POST /generate、input_ids 与 SSE 流式响应。Ray 2.56.0 安装在同一环境，本次单卡服务的 use_ray=false。

组别	输入结构	测量前准备
共享前缀	2048 共同 + 64 独立	flush 后先发 1 条同前缀预热
分散前缀	2112 全部独立，首 token 各异	flush 后直接测量

两组各 32 条测量请求，最大并发均为 8；输入均为 2112 token。temperature=0、max_new_tokens=16、ignore_eos=true、sampling_seed=2026。请求用固定随机种子 2026 构造，组内顺序固定。共享组预热请求与测量请求有相同的 2048 token 前缀，64 token 后缀不同；预热请求不计入统计。

先用 4 token 短请求完成服务预热。每组测量前，上一组请求全部结束后调用 POST /flush_cache?timeout=30；脚本校验 HTTP 200 与“ Cache flushed. ”，然后才继续。运行顺序为分散组、共享组；每组计时窗口从提交 32 条请求开始，到最后一条流结束为止。

指标口径与可追溯数据

缓存命中率 = sum(cached_tokens) / sum(prompt_tokens)；实际 Prefill token = sum(prompt_tokens - cached_tokens)。吞吐量 = 测量组总输出 token / 组墙钟时间。TTFT 从发起请求计到首个含生成 token 的 SSE 事件；端到端延迟计到 [DONE]；TPOT = (端到端延迟 - TTFT) / (输出 token 数 - 1)。p50、p95 使用对逐请求值做线性插值。

results/target1/ 各组的 requests.jsonl 保存 input_ids 和参数；responses.jsonl 保存每条请求的 token 数、命中数、计时及状态；summary.json 与 comparison.json 对应下页主表。flush.json、prewarm.json、service_warmup.json 可核验实验顺序。

任务一：结果与分析

指标	分散前缀	共享前缀
成功率	32/32 (100%)	32/32 (100%)
输出吞吐量 (token/s)	116.98	609.45
缓存命中率	0.00%	96.97%
实际 Prefill token	67,584	2,048
TTFT p50 / p95 (ms)	576.0 / 1189.8	42.2 / 251.6
TPOT p50 / p95 (ms)	25.0 / 47.7	7.8 / 9.6
端到端 p50 / p95 (ms)	946.8 / 1880.6	164.3 / 361.1

两组输入合计均为 67,584 token，输出合计均为 512 token；共享组命中 65,536 token（每条 2048），仅需执行 32×64 =2,048 token 的 Prefill。分散组首 token 各异，命中 0。吞吐量从 116.98 增至 609.45 token/s，主要来自共享组大幅缩短的 Prefill。

为什么相同前缀可复用

因果自注意力为 Attention(Q,K,V) = softmax(Q K^T / sqrt(d_k) + causal_mask) V，且每层位置 i 有 K_i = h_i W_K、V_i = h_i W_V。因果掩码使位置 i 的隐状态只依赖位置 1 到 i 的 token；同一模型、同一 token 序列及位置的前缀，其逐层 h_i 与 K/V 相同，后续独立后缀不会改变已算好的前缀 K/V。RadixCache 命中后提供相应 KV 索引，Prefill 只计算未命中的 64 token 后缀；新生成的 16 token 仍需逐步 Decode。

TTFT 包含排队与 Prefill，因此共享组 p50 从 576.0 ms 降至 42.2 ms，p95 从 1189.8 ms 降至 251.6 ms。TPOT 所对应的 Decode 计算本身没有被前缀缓存免除；本次 TPOT 也下降（p50 25.0→7.8 ms），可由较短 Prefill 减少 GPU 争用、排队及调度干扰解释，不能据此推断单个 Decode token 的算术量减少。端到端延迟同时受 TTFT 与后续生成影响。

任务二：/generate 请求主流程

HTTP 接收 / SSE	entrypoints/http_server.py: generate_request（约 769 行）
↓	
请求解析与派发	managers/tokenizer_manager.py: generate_request / _send_one_request（576 / 1319 行）
↓	
进入等待队列	managers/scheduler.py: event_loop_normal / handle_generate_request / _add_request_to_queue（1505 / 1998 / 2258 行）
↓	
前缀匹配与组批	schedule_batch.py: Req.init_next_round_input（1123）；schedule_policy.py: match_prefix_for_req（85）；radix_cache.py: match_prefix（358）
↓	
Prefill 预算 / 执行	managers/schedule_policy.py: PrefillAdder.add_one_req（858 行）；schedule_batch.py: prepare_for_extend（2008 行）
↓	
模型执行：Prefill / Decode	managers/scheduler.py: run_batch（3145 行）；managers/tp_worker.py: forward_batch_generation（482 行）
↓	
缓存更新与流式输出	mem_cache/radix_cache.py: cache_finished_req / cache_unfinished_req（438 / 485 行）→ tokenizer_manager.py: _wait_one_response（1425 行）

主流程说明

TokenizerManager 将 input_ids 组成 TokenizedGenerateReqInput 后发给 Scheduler；event_loop_normal 从接收器取请求，process_input_requests → handle_generate_request 构造 Req，_add_request_to_queue 将普通请求追加至 waiting_queue。

get_new_batch_prefill 读取等待队列。当前 FCFS 路径中，Req.init_next_round_input 直接调用 RadixCache.match_prefix，返回已缓存前缀的 KV 索引；schedule_policy.match_prefix_for_req 是缓存感知调度的辅助入口。extend_input_len = 全部待处理 token - 命中前缀长度。PrefillAdder 按实际延展长度计预算，ScheduleBatch.prepare_for_extend 仅打包未命中的 token。

run_batch 调用 model worker 执行 Prefill 与后续 Decode。未结束请求经 maybe_cache_unfinished_req、结束请求经 release_kv_cache，分别调用 RadixCache.cache_unfinished_req / cache_finished_req，把已提交 token 的 KV 索引插入树并管理引用。Scheduler 输出经 TokenizerManager._wait_one_response 逐次 yield，由 HTTP 层编码成 data: ... 的 SSE 事件，最后发送 [DONE]。

上述相对路径以 SGLang 0.5.14 安装包的 sglang/srt/ 为根；函数名与行号可在该版本源码中核对。

四：AI 使用说明情况

使用的模型、提示词、AI 如何帮助学习、是否产生误导及如何核验。
gpt6.1sol

五：作业感受

（1）如何完成第二次挑战：AI 使用方式与工作迭代等。

使用codex完成

（2）最困难的部分及克服方法。

最困难的部分还是理解知识代码等

（3）从 0 到 1 的科研工作需要哪些研究、对今后科研的启发。

了解理论知识，知道原理，才能更好的实践